Why is JAVA a platform independent language?

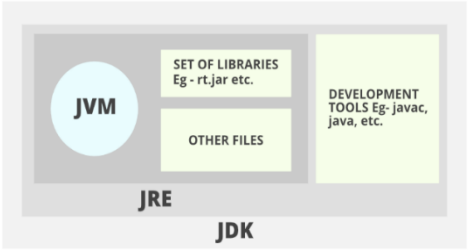
Java is platform-independent because it compiles code into bytecode, which is executed by the Java Virtual Machine (JVM). The JVM is available for multiple platforms, allowing the same bytecode to run on any device with a compatible JVM, enabling the "write once, run anywhere" principle. The only condition to run that byte code is for the machine to have a runtime environment (JRE) installed in it.

Why JAVA is not a purely Object-oriented language?

Java is not purely object-oriented because it includes primitive data types (e.g., int, char, boolean) that are not objects. These primitives provide a way to optimize performance and memory usage, contrasting with purely object-oriented languages where everything, including basic data types, is treated as an object.

JVM vs JRE vs JDK

	JDK (Java Development Kit)	**JRE (Java Runtime Environment)**	**JVM (Java Virtual Machine)**
Purpose	Provides tools for Java development.	Provides the runtime environment for executing Java applications.	Executes Java bytecode.
Components	Includes the JRE, compilers, debuggers, and other development tools.	Includes the JVM, core libraries, and supporting files.	Includes a bytecode interpreter and a Just-In-Time (JIT) compiler.
Role	Used by developers to write, compile, and debug Java applications.	Allows users to run Java applications but does not include development tools.	Provides platform independence by running Java programs on any device with a JVM.

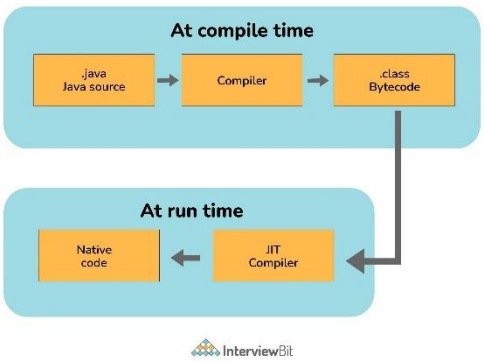


JIT Complier

JIT stands for Just-In-Time and it is used for improving the performance during run time. It does the task of compiling parts of byte code having similar functionality at the same time thereby reducing the amount of compilation time for the code to run.

The compiler is nothing but a translator of source code to machine-executable code. But what is special about the JIT compiler? Let us see how it works:

First, the Java source code (.java) conversion to byte code (.class) occurs with the help of the javac compiler.



Then, the .class files are loaded at run time by JVM and with the help of an interpreter, these are converted to machine understandable code.

JIT compiler is a part of JVM. When the JIT compiler is enabled, the JVM analyses the method calls in the .class files and compiles them to get more efficient and native code. It also ensures that the prioritized method calls are optimized.

Once the above step is done, the JVM executes the optimized code directly instead of interpreting the code again. This increases the performance and speed of the execution.

Class Loader

Java Class loader is the program that belongs to JRE (Java Runtime Environment). The task of Class Loader is to load the required classes and interfaces to the JVM when required.

Example- To get input from the console, we require the scanner class. And the Scanner class is loaded by the Class Loader.

How JAVA is different from C++?

- **Platform Independence:** Java compiles to bytecode, runs on JVM, making it platform-independent while C++ compiles to machine code specific to the operating system and hardware.

- **Memory Management:** Java has automatic garbage collection.

- Java has no pointers, no multiple inheritance (supports interfaces), includes built-in thread support.

- Java has rich standard library with extensive built-in functionality.

- **Application Domains:** Java is commonly used for web applications, enterprise solutions, and Android apps on the other hand C++ is used for system/software development, game development, and applications requiring high performance.

Why JAVA does not use pointers unlike C++?

Pointers are quite complicated and unsafe to use by beginner programmers. Java focuses on code simplicity, and the usage of pointers can make it challenging. Pointer utilization can also cause potential errors. Moreover, security is also compromised if pointers are used because the users can directly access memory with the help of pointers.

Thus, a certain level of abstraction is furnished by not including pointers in Java. Moreover, the usage of pointers can make the procedure of garbage collection quite slow and erroneous. Java makes use of references as these cannot be manipulated, unlike pointers.

POPS vs OOPS

- POPS (Procedure-Oriented Programming System) emphasizes functions and procedures while OOPS (Object-Oriented Programming System) emphasizes objects that encapsulate data and methods.

- In POPS programs are divided into small procedures or functions while in OOPS into classes and objects.

- In POPS data and functions are separate, data is passed between functions. In OOPS data and methods are bundled together; objects interact through methods.

- POPS: C, Fortran. OOPS: JAVA, C++, Python.

Advantages of OOPS

- Better representation of real-world scenario.

- Enhanced security.

- Code reusability & easy code modification.

- Better software maintenance.

Pillars of OOPS

Encapsulation (Security), Abstraction (Hides Complexities), Inheritance (Reusability) and Polymorphism (Simplicity).

Class and Objects

Class: It's a template for creating objects consists with its properties (attributes) and behaviours (methods).

Object: Instance of a class.

CLASS

```
class Car {
        String colour;
        void start() {}
}
```

OBJECT

```
class UseCar {
        public static void main(string[] args) {
                Car car1 = new Car();
                car1.colour = "Black";
                car1.start();
        }
}
```

JAVA packages

Java packages organize classes and interfaces. Declare a package at the top of a Java file using "package" keyword. Use "import" to access classes from other packages.

```
package com.example.carproject;
public class MyCar {
        // Class implementation
}
```

```
import com.example.myproject.carproject;
```

Data members and methods

Data members are variables within a class that store data, while methods are functions that define behaviours or operations that can be performed on the class's data. In the above example of Car class colour are the data members while start() is a method.

Methods vs Functions

Methods are functions defined within classes in object-oriented programming, operating on class data. Functions are standalone units of code in procedural programming.

Difference between Heap and Stack memory in JAVA? And how to utilize this.

Stack memory is the portion of memory that was assigned to every individual program. And it was fixed. On the other hand, Heap memory is the portion that was not allocated to the java program but it will be available for use by the java program when it is required, mostly during the runtime of the program.

Java Utilizes this memory as: When we write a java program then all the variables, methods, etc are stored in the stack memory. And when we create any object in the java program then that object was created in the heap memory. And it was referenced from the stack memory.

Instance vs Local Variable

Instance variables are those variables that are accessible by all the methods in the class. They are declared outside the methods and inside the class. These variables describe the properties of an object and remain bound to it at any cost. All the objects of the class will have their copy of the variables for utilization. If any modification is done on these variables, then only that instance will be impacted by it, and all other class instances continue to remain unaffected.

Local variables are those variables present within a block, function, or constructor and can be accessed only inside them. The utilization of the variable is restricted to the block scope. Whenever a local variable is declared inside a method, the other class methods don't have any knowledge about the local variable.

```
class Car {
        String colour;                          // Here "colour" is instance variable
        void Car(String setColor) {             // and "setColor" is a local variable
                colour = setColor;
        }
}
```

Default instance variable values

Reference: null, Integer: 0, Float: 0.0, Boolean: false

Equality(==) operator vs equals() method

We are already aware of the (==) equals operator. That we have used this to compare the equality of the values. But when we talk about the terms of object-oriented programming, we deal with the values in the form of objects. And this object may contain multiple types of data. So, using the (==) operator does not work in this case. So, we need to go with the .equals() method.

Both [(==) and .equals()] primary functionalities are to compare the values, but the secondary functionality is different.

So, in order to understand this better, let's consider this with the example –

```
String str1 = "CAR";
String str2 = "CAR";
System.out.println(str1 == str2);
// True
```

```
String str1 = new String("CAR");
String str2 = "CAR";
System.out.println(str1 == str2);
// False
```

```
String str1 = new String("CAR");
String str2 = "CAR";
System.out.println(str1.equals(str2));
// True
```

First code will print true. We know that both strings are equals so it will print true. But here (==) Operators don't compare each character in this case. It compares the memory location. And because the string uses the constant pool for storing the values in the memory, both str1 and str2 are stored at the same memory location.

Now, in the second code it will print false. Because here no longer the constant pool concepts are used. Here, new memory is allocated. So here the memory address is different, therefore (==) Operator returns false. But the twist is that the values are the same in both strings. So how to compare the values? Here the .equals() method is used.

.equals() method compares the values and returns the result accordingly. In the third code it returns true.

Entity vs Driver Class

In Java, an entity class represents a real-world entity with attributes and behaviours, often mapped to a database table in object-relational mapping (ORM). A driver class, on the other hand, typically contains the main method and is used to instantiate and interact with objects of other classes, such as entity classes.

In the above example Car is an Entity class whereas UseCar is a driver class.

Static method, variables and class in JAVA

"static" keyword is a non-access modifier used for methods attributes as well as inner class (class within the class). Static methods/attributes can be accessed without creating an object.

Static variable belongs to the class rather than instances that shared among all objects and stored in method area of RAM. It can be accessed by class_name.attribute_name. The static variable is initialized when the class first object of that class is created or a static value is called and stored. Ex Math.pi.

Similarly static method is used to access a method of a class without creating its object and can access only static data members.

```
class Car {
        static String brand = "BMW";
        static void display() {
                System.out.println(brand);
        }
}
```

```
class UseCar {
        public static void main() {
                System.out.println(Car.brand);
                Car.display();
        }
}
```

Why main method is static in JAVA?

The main method in Java is static to allow it to be called without creating an instance of the class, enabling program execution without object instantiation.

Access Modifiers in JAVA

	public	protected	default	Private
Same Class	YES	YES	YES	YES
Same Package Subclass	YES	YES	YES	NO
Same Package Non-Subclass	YES	YES	YES	NO
Different Package Subclass	YES	YES	NO	NO
Different Package Non-Subclass	YES	NO	NO	NO

Wrapper class in JAVA

Wrapper classes in Java provide a way to use primitive data types (such as int, char, etc.) as objects. They are part of the "java.lang" package and include classes like Integer, Character, Double, Boolean, etc. Wrapper classes are useful for converting primitive types to objects and vice versa.

```
Integer integer = new Integer(10);
```

Encapsulation

Encapsulation is the process of binding or wrapping the data and the methods that operates on the data into a single entity by which the data can be prevented from being accessed by the code outside this entity. According to encapsulation the data members should be kept in private while the methods are in public, by which user can only access the methods not the data members. For example. We can view, deposit and withdraw some amount from the bank but we cannot access the private data from the bank.

WITHOUT ENCAPSULATION

```
class Account {
        int balance;
        int getBalance() {
                return balance;
        }
}
```

WITH ENCAPSULATION

```
class Account {
        private int balance;
        public int getBalance() {
                return balance;
        }
}
```

Abstraction

Abstraction is a method that shows only essential attributes and hides unnecessary information. The main purpose of abstraction is to make interaction with the application simple. Its just like you can find your bank balance by calling a method but internally how the method fetch your bank balance from the server that is hides from user.

```
class Account {
        int balance;
        int fetchBalanceFromServer() {      // Fetch Balance       }
        int getBalance() {
                return fetchBalanceFromServer();                    // Abstraction
        }
}
```

Constructor

A constructor is a special method of a class that automatically created as a new object of that class is created. It has a same name as that of the class and don't have any return type.

```
class Car {
        private colour;
        public Account(String colour) {
                this.colour = colour;
        }
}
```

PARAMETERIZED CONSTRUCTOR

```
class UseCar {
        public static void main() {
                Car car1 = new Car("Black");
        }
}
```

Default Constructor

Default constructor is automatically inserted by java compiler, if we have not created any constructor explicitly with no arguments and empty body.

Copy Constructor

Copy Constructor is the constructor used when we want to initialize the value to the new object from the old object of the same class.

```
class Car{
        String colour;
         Car(Car c){
                this.colour = c.colour;
        }
}
```

Getter and Setter methods

Getter & setter are the methods defined to work upon instance variable of the class. Getter method returns its value while setter method updates its value.

```
class Car{
        private String colour;
         public string getColour(){
                return colour;
        }
        public void setColour(String colour) {
                this.colour = colour;
        }
}
```

Method Overloading

Overloading is a concept in which we can create multiple versions of same entity in same scope. So, a class can multiple methods having same name but different parameters. For example, Arrays.sort() sorts array of integer, float, char etc.

Constructor overloading

Constructor overloading is the process of creating multiple constructors in the class consisting of the same name with a difference in the constructor parameters. Depending upon the number of parameters and their corresponding types, distinguishing of the different types of constructors is done by the compiler.

Ways to implement method overloading

```
public class Add {
        public double add(int a, double b) {
                return a + b;
        }
        public double add(double a, double b) {          // By changing types of parameters.
                return a + b;
        }
        public int add(int a, double b, double c) {      // By changing number of parameters.
                 return a + b + c;
        }
        public double add(double a, int b) {             // By changing order of parameters.
                return a + b;
        }
}
```

Can the main method be overloaded?

Yes, It is possible to overload the main method. We can create as many overloaded main methods we want. However, JVM has a predefined calling method that JVM will only call the main method with the definition of -

```
public static void main(string[] args) {}
```

For Example:

```
class Main {
        public static void main(String args[]) {                    // Actual main()
                System.out.println(" Main Method");
        }
        public static void main(int[] args){                        // Overloaded main()
                System.out.println("Overloaded Integer array Main Method");
        }
}
```

"this" keyword

"this" is a special object reference which is also a keyword. "this" is automatically created by Java in a method argument, as we call that method. "this" stores the memory address of current object. With the help of "this" keyword we can resolve overlapping of local and instance variable. For example, refer example of constructor.

Inheritance

Inheritance is a technique using which we can acquire the features of an existing class/object in a newly created class or object. The main benefits of Inheritance is reusability and maintainability. Inheritance is implemented using “extends” keyword. There are mainly five types of Inheritance i.e. Single, *Multiple, Multilevel, Hierarchical and Hybrid Inheritance.

```
class Vehicle {                         // PARENT CLASS
        void startEngine() {
                // Start Engine
        }
}
class Car() extends Vehicle {       // CHILD CLASS
        void playMusic() {
                // Play Music
        }
}
```

PARAMETERIZED CONSTRUCTOR

```
class UseCar() {
        static void main() {
                Car car = new Car();
                car.startEngine();
                car.playMusic();
        }
}
```

Why multiple inheritance is not supported in JAVA

Java does not support multiple inheritance to avoid the complexities and ambiguities (Diamond Problem) it introduces, promoting simpler and more maintainable code. Instead, Java uses interfaces to achieve multiple inheritance of type.

Diamond Condition: When a class inherits from two classes that have a common ancestor, it creates ambiguity about which inherited properties or methods to use. For example:

```
class A {
        void display() {
                System.out.println("A's display");
        }
}
class B {
        void display() {
                System.out.println("B's display");
        }
}
// Hypothetical scenario where Java allows multiple inheritance
class C extends A, B {
        // Ambiguity: Which display() method should C inherit?
}
```

MULTIPLE INHERITANCE USING INTERFACES

```
interface A {
        void display();
}
interface B {
        void display();
}
class C implements A, B {
        public void display() {
                System.out.println("C’s Display");
        }
}
```

super keyword

The super keyword is used to access hidden fields and overridden methods or attributes of the parent class.

Following are the cases when this keyword can be used:

- Accessing data members of parent class when the member names of the class and its child subclasses are same.
- To call the default and parameterized constructor of the parent class inside the child class.
- Accessing the parent class methods when the child classes have overridden them.

```
class Vehicle {
        private String vehicleName;
        Vehicle(String vehicleName) {
                this.vehicleName = vehicleName;
        }
        public void start() {
                // Start Vehicle
        }
}
```

```
class Car extends Vehicle {
        private String vehicleName;
        Child(String vehicleName) {
                super("Car");                   // Case II
                this.vehicleName = vehicleName;
        }
        public void start(){          // method overriding
                // Start Car
                Sout(super.vehicleName);      // Case I
                super.start();                  // Case III
        }
}
```

method overriding

Whenever a child class contains a method with the same prototype as the parent class, then we say that the method of child class has overridden the method of parent class. For Example, refer example of "super keyword".

Why we override a super class method?

To provide better/correct implementation of that method in child class.

Let's understand with an example of "super" keyword. In this example the start() method is overridden because the mechanism to start a car can be different from a vehicle and these different functionalities are not in vehicle class start() method that's why we override the start() method on Car class to explicitly add these functionalities.

Can static as well as private methods be overridden?

The statement in the context is completely False. The static methods have no relevance with the objects, and these methods are of the class level. In the case of a child class, a static method with a method signature exactly like that of the parent class can exist without even throwing any compilation error.

The phenomenon mentioned here is popularly known as method hiding, and overriding is certainly not possible. Private method overriding is unimaginable because the visibility of the private method is restricted to the parent class only. As a result, only hiding can be facilitated and not overriding.

Method overloading vs overriding

Method Overloading	Method Overriding
Overloading can be done either within the same or between methods of parent & child class.	Overriding can never be done within a single class and it always requires inheritance i.e. overriding can be done between of parent & child only.
Overloading says methods must have same name but with different arguments.	Overriding says methods must have exactly same prototype. Although overriding allows covariant return types

"final" class, method and variable

In Java, the final keyword is used as defining something as constant /final and represents the non-access modifier.

final variable: When a variable is declared as final in Java, the value can't be modified once it has been assigned. If any value has not been assigned to that variable, then it can be assigned only by the constructor of the class.

final method: A method declared as final cannot be overridden by its children's classes. A constructor cannot be marked as final because whenever a class is inherited, the constructors are not inherited. Hence, marking it final doesn't make sense. Java throws compilation error saying - modifier final not allowed here

final class: No classes can be inherited from the class declared as final. But that final class can extend other classes for its usage. Some predefined final class provide by java are Math, String, All wrapper classes like (Integer, Character etc).

final vs finally vs finalize

All three keywords have their own utility while programming.

Final: If any restriction is required for classes, variables, or methods, the final keyword comes in handy. Inheritance of a final class and overriding of a final method is restricted by the use of the final keyword. The variable value becomes fixed after incorporating the final keyword.

Finally: It is the block present in a program where all the codes written inside it get executed irrespective of handling of exceptions.

Finalize: Prior to the garbage collection of an object, the finalize method is called so that the clean-up activity is implemented.

```
final int a=100;
a = 0;  // error
```

```
try {
        int variable = 5;
}
catch (Exception exception) {
        Sout(exception);
}
finally {
        Sout("Finally block");
}
```

```
public static void main() {
        String s = new String("Hi!");
        example = null;
        // Garbage collector called
        System.gc();
}
public void finalize() {
        // Finalize called
}
```

is it possible that finally block will not be executed?

Yes. It is possible if we use System.exit() or if there are fatal errors like Stack overflow, memory access error etc.

Binding

When compiler decides that which function call which function body will get executed. There are two types of binding:

Early/ Static/ Compile Time Binding: If the method being called is a static method, then java selects the method by looking at the (class) of the reference used in the call and not the type of object to which the reference is pointing.

Late/ Dynamic/ Run Time Binding: If the method being called is a non-static method i.e. instance method, then Java selects the method by considering the type of object, pointed by the reference and not considering the type of reference itself.

EARLY BINDING

```
class Parent {
        public static void show() {}
}
class Child {
        public static void show() {}
}
class Main {
        public static void main() {
                Parent p = new Child();
                p.show();
        }
}
```

LATE BINDING

```
class Parent {
        public void show() {}
}
class Child {
        public void show() {}
}
class Main {
        public static void main() {
                Parent p = new Child();
                p.show();
        }
}
```

Polymorphism

Polymorphism means the ability to acquire multiple forms. There are two types of polymorphism:

Compile Time/ Static Polymorphism: We must have different implementations of the same method in a class i.e. Method Overloading.

Run Time/ Dynamic Polymorphism: We must have able to call different versions of the same method using the same reference variable. These methods should be present in child class and the reference must be of parent class.

```
class Main {                                    // Run Time Polymorphism
        public static void main() {
                Parent p = new Parent();
                p.show();                       // call show() method of parent class
                p = new Child();
                p.show();                       // call show() method of child class
        }
}
```

Abstract method and class

When There are situations when we have a method in super class which cannot be implemented properly due to lack of information in super class. But we need this method for achieving runtime polymorphism and thus in such cases we must declare a method as abstract.

- To make a method abstract it is compulsory to use the keyword “abstract” in the method prototype.
- An abstract method should never have any implementation in the class where it is being declared.
- If a method has been declared as “abstract” in the class, then the class itself must be prefixed with the keyword abstract.
- If a class is abstract then we are not allowed to create its “object”. Although we can create reference of the class.
- If a class inherits an abstract class, then either it must compulsorily override all the abstract methods inherited from the super class or the child class itself will have to be prefix with the keyword abstract and we won’t be allowed to create its object also.

```
abstract class Instrument {
        abstract public void sound();
}
class Violin extends Instrument {
        public void sound() {
                // Violin Sound
        }
}
class Guitar extends Instrument {
        public void sound() {
                // Guitar Sound
        }
}
```

```
class UseInstrument {
        public static void main() {
                Instrument i;
                i = new Violin();
                i.sound();
                i = new Guitar();
                i.sound();
        }
}
```

Which methods cannot be declared as abstract?

- **Static methods:** Abstract methods cannot be static because static methods belong to the class, not instances.
- **Final methods:** Abstract methods cannot be final because final methods cannot be overridden.
- **Private methods:** Abstract methods cannot be private because private methods are not visible to subclasses.
- **Constructors:** Constructors cannot be abstract because they are used to instantiate objects, and abstract methods do not have a body or implementation.

Interface

An interface is almost same as a pure abstract class. Just like a class or abstract class an interface also can contain data members. But every data declared in an interface is automatically converted to “public” “static” and “final” by java. An interface can also contain methods but every method by default is “public” and “abstract”. However, from java 8, an interface can also contain “default” and “static” methods.

Just like we cannot instantiate an abstract class, similarly we also cannot instantiate an interface. However, we can create a reference of an interface.

Just like an abstract class an interface also can be inherited by child classes but the keyword used for this inheritance is implements.

The reference of an interface can point to the object of its implementation class or child class. And this forms the basis of runtime polymorphism. If a class inherits an interface, then it is compulsory to override every abstract method inherited from the interface. Otherwise, the derived class also will have to be declared as abstract.

Implementation of interfaces are present at “Why multiple inheritance is not supported in JAVA?” question.

Abstract vs interface

Availability of methods: Only abstract methods are available in interfaces, whereas non-abstract methods can be present along with abstract methods in abstract classes.

Variable types: Static and final variables can only be declared in the case of interfaces, whereas abstract classes can also have non-static and non-final variables.

Inheritance: Multiple inheritances are facilitated by interfaces, whereas abstract classes do not promote multiple inheritances.

Data member accessibility: By default, the class data members of interfaces are of the public- type. Conversely, the class members for an abstract class can be protected or private also.

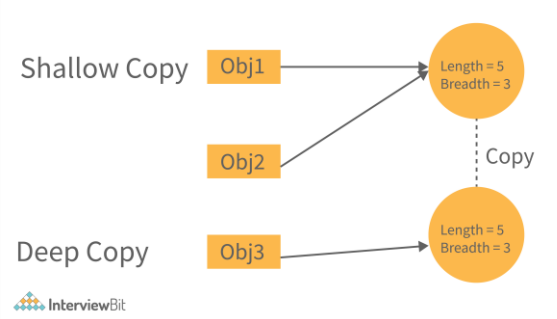
Implementation: With the help of an abstract class, the implementation of an interface is easily possible. However, the converse is not true.

Sallow vs deep copy in JAVA

Shallow copy - The shallow copy only creates a new reference and points to the same object.

Now by doing this what will happen is the new reference is created with the name obj2 and that will point to the same memory location.

Deep Copy - In a deep copy, we create a new object and copy the old object value to the new object.




```
Rectangle obj2 = obj1;
```

```
Rectangle obj3 = new Rectangle();
Obj3.length = obj1.length;
Obj3.breadth = obj1.breadth;
```

Garbage collector

The garbage collector in Java automatically reclaims memory by deallocating objects that are no longer in use. It identifies unreferenced objects and frees their memory, improving memory management and preventing memory leaks. For example, refer. “Finalize” question.

Which part of memory(stack or heap) is cleaned in garbage collection process

Heap

JAVA exception handling

Java exception handling provides a robust mechanism for handling runtime errors, ensuring that the program can handle unexpected situations gracefully without crashing. It uses a combination of try, catch, finally, and throw blocks to manage exceptions.

```
public class ExceptionHandlingExample {
	public static void main() {
		try {
			int result = divide(10, 0);
			System.out.println("Result: " + result);
		} catch (ArithmeticException e) {
			System.out.println("Error: " + e.getMessage());
		} finally {
			System.out.println("Finally block executed");
		}
	}
	public static int divide(int a, int b) throws ArithmeticException {
		return a / b;
	}
}
```

Can a single try block and multiple catch co-exist in JAVA program

Yes, multiple catch blocks can exist but specific approaches should come prior to the general approach because only the first catch block satisfying the catch condition is executed. The given code illustrates the same:

```
public class MultipleCatch {
	public static void main() {
		try {
			int n = 1000, x = 0, arr[] = new int[n];
			for (int i = 0; i <= n; i++) {}
		} catch (ArrayIndexOutOfBoundsException exception) {   // If x != 0, then this block will get executed
			System.out.println("1st block = ArrayIndexOutOfBoundsException");
		} catch (Exception exception) {                        // This block will get executed because x = 0
			System.out.println("3rd block = Exception");
		}
	}
}
```

Types of relationships

- **Is a relationship / Inheritance:** An IS-A relationship signifies that one object is a type of another.
- **Has a relationship / Association:** A HAS-A relationship signifies that a class is associated with that is it holds object(s) of another class in its body.

Association

- **Aggregation:** In aggregation, the component object can exist independently of the container object.
  Example: a car object "has-a" music player object.
  Implementation: By creating the object of MusicPlayer in Car class and call its methods in Car class.
- **Composition:** Means contained object cannot exist without the owner or container object.
  Example: A College is a composition of Department. If the College object doesn't exist or dies then Department object will also not exist.
  Implementation: Using Inner class.